Software Requirements Specification (SRS)
Design and Implementation of a Distributed, Sharded Transaction Engine with Atomic Cross-Shard Transfers

1. Introduction

This document specifies the functional and non-functional requirements for the Distributed, Sharded Transaction Engine. The system is designed to support atomic cross-shard money transfers under failures, explicitly implementing distributed systems protocols rather than relying on existing databases.

1.1 Purpose

The purpose of this SRS is to provide a complete and unambiguous specification of system behavior, interfaces, constraints, and quality attributes for developers, evaluators, and maintainers.

1.2 Scope

The system provides a fault-tolerant distributed transaction engine inspired by real-world payment infrastructures. It supports application-level sharding, two-phase commit, write-ahead logging, deterministic concurrency control, and automatic crash recovery.

1.3 Definitions and Acronyms

Shard: A storage node responsible for a subset of users.
WAL: Write-Ahead Log, an append-only durable log.
2PC: Two-Phase Commit protocol.
Coordinator: Node responsible for orchestrating distributed transactions.

2. Overall Description

The system is composed of clients, a transaction coordinator, and multiple sharded storage nodes. Components communicate via RPC and can be deployed independently.

2.1 Product Functions

The system supports user account management, atomic money transfers, crash-safe persistence, deterministic execution, and automated recovery.

2.2 Operating Environment

The system runs on Linux, is containerized using Docker, and communicates over TCP/IP networks.

2.3 Constraints

The system must not rely on external databases for correctness. All atomicity and recovery logic must be implemented internally.

3. System Architecture

Clients submit requests through a load balancer to a transaction coordinator, which orchestrates distributed transactions across sharded storage nodes.

3.1 Sharding Model

Users are deterministically mapped to shards using consistent hashing. Shards can be added dynamically with minimal data redistribution.

4. Functional Requirements

The system shall support millions of users, process cross-shard transfers, implement two-phase commit, persist all changes via WAL, and recover safely from crashes.

5. Non-Functional Requirements

The system guarantees atomicity, durability, scalability, fault tolerance, and maintainability under concurrent load.

6. Non-Goals

The system does not provide SQL support, global external consistency, or distributed query optimization.

7. Future Enhancements

Future work includes coordinator replication, shard replication, WAL compaction, snapshotting, and observability metrics.